

DevOps Bash Automation

Task Guide for Sysadmins with AWS Experience

You already have the foundation — sysadmin skills, AWS knowledge, and Bash basics. This guide maps out the most impactful automation tasks you can build, organized by category.

1. Infrastructure & System Health

Disk / CPU / Memory Alerts

Monitor thresholds and send notifications before services fail.

```
df -h | awk '$5+0 > 80 {print $6" is "$5" full"}' | mail -s "Disk Alert" ops@company.com
```

Log Rotation & Cleanup

Auto-compress logs older than N days and purge beyond retention period.

```
find /var/log -name '*.log' -mtime +7 -exec gzip {} \;
```

Automated Backup Verification

Don't just create backups — test restores on a schedule and alert on failure.

```
tar -tzf backup.tar.gz > /dev/null && echo 'OK' || echo 'CORRUPT'
```

2. AWS-Specific Automation

Cost Reports

Pull EC2/S3 spending daily and email a summary to your team.

```
aws ce get-cost-and-usage --time-period Start=2024-01-01,End=2024-01-31 \ --granularity MONTHLY --metrics BlendedCost
```

Snapshot Lifecycle Management

Auto-create EBS snapshots and delete ones older than 30 days to control costs.

```
aws ec2 describe-snapshots --owner-ids self \ --query 'Snapshots[?StartTime<=`2024-01-01`].SnapshotId'
```

Auto Start/Stop Dev Instances

Schedule non-prod instances to shut down nights and weekends — big cost savings.

```
aws ec2 stop-instances --instance-ids $(aws ec2 describe-instances \ --filters 'Name=tag:Env,Values=dev' --query 'Reservations[].Instances[].InstanceId' \ --output text)
```

IAM Audit Script

Find unused IAM users, access keys older than 90 days, and over-permissioned roles.

```
aws iam list-users --query 'Users[?PasswordLastUsed==null].UserName'
```

3. Deployment & Release

Zero-Downtime Deployment

Pull new code, build, swap symlink, reload service — no downtime.

```
git pull origin main && npm run build \ && ln -sf /app/releases/new /app/current \ && systemctl reload nginx
```

One-Command Rollback

Instantly revert to the previous release if something goes wrong.

```
ln -sf /app/releases/previous /app/current && systemctl reload nginx
```

Environment Sync (Prod → Staging)

Copy sanitized production DB to staging so devs test against real data shapes.

```
mysqldump prod_db | sed 's/real@email/test@test.com/g' | mysql staging_db
```

4. Monitoring & Alerting

Service Health Checks

Ping your endpoints every minute and post to Slack if any are down.

```
curl -sf https://myapp.com/health || \ curl -X POST $SLACK_HOOK -d '{"text":"myapp DOWN!"}'
```

SSL Certificate Expiry Warnings

Alert 30, 7, and 1 day before cert expiry — never let SSL lapse.

```
echo | openssl s_client -connect myapp.com:443 2>/dev/null \ | openssl x509 -noout -dates
```

Failed Login Report

Parse auth logs nightly and email a summary of suspicious activity.

```
grep 'Failed password' /var/log/auth.log | awk '{print $11}' \ | sort | uniq -c | sort -rn | head -20
```

Where to Start — Priority Matrix

Task	Effort	Impact	Start Here?
------	--------	--------	-------------

Disk/CPU Alerts	Low	High	✓ Yes
Auto Stop Dev Instances	Low	High (saves \$)	✓ Yes
SSL Cert Expiry Warnings	Low	High	✓ Yes
IAM Audit Script	Medium	High	✓ Yes
Backup Verification	Medium	Very High	✓ Yes
Zero-Downtime Deploy	High	High	After basics
Env Sync Prod→Staging	High	Medium	After basics

Pro Tip: Pick the task that currently causes you the most pain — something you do manually every week. That's your first script. Once it runs automatically, you'll never look back.